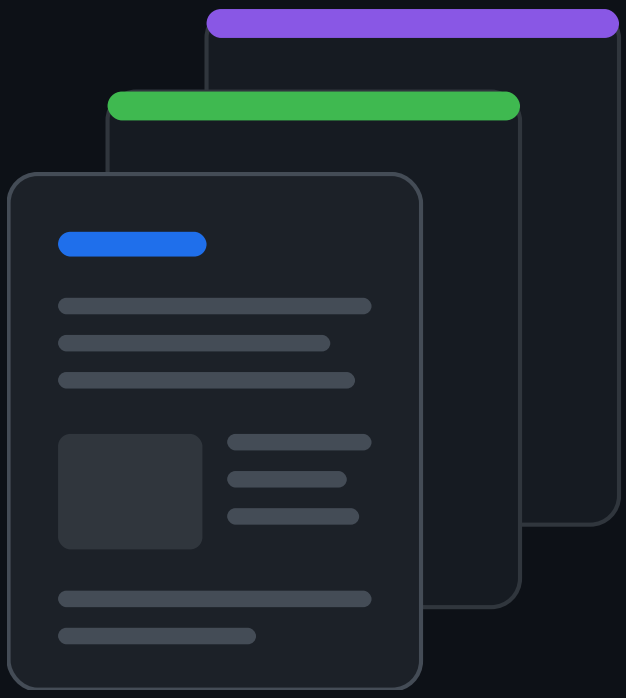


genpdf

Manual de instalación, uso y arquitectura del generador de documentos y presentaciones en PDF

AUTOR

S4nti4goCoder

Contenido

Qué es genpdf 4

Cómo funciona 5

Motores de impresión 5

Requisitos 6

Instalación 7

1. Clonar el repositorio 7
2. Crear y activar el entorno virtual 7
3. Instalar las librerías de Python 7
4. Instalar Pango 8
5. Verificar la instalación 9

Uso 10

Comandos 10

Opciones 10

Resultado del comando 11

Flujo de trabajo 11

Escribir un documento 12

Frontmatter 12

Bloques disponibles 13

Escribir una presentación 14

Frontmatter 14

Reglas generales 14

Tipos de diapositiva 14

Límites de contenido 15

Iconos 15

Plantillas 16

Plantillas de contenido 16

Plantillas de diseño 16

Validaciones 18

Antes de generar: `check` 18

Después de generar: `build` 18

Estructura del proyecto 19

Carpeta `content` 19

Carpeta `docs` 19
Carpeta `genpdf` 19
Carpeta `tests` 22

Desarrollo y extensión **23**
Comandos de desarrollo 23
Cómo extender el proyecto 23

Configuración y solución de problemas **25**
Variables de entorno 25
Solución de problemas 25
Limitaciones conocidas 25

Autor **27**

Qué es genpdf

Generador de documentos y presentaciones en PDF a partir de archivos Markdown.

El contenido se escribe en un archivo `.md` y un comando lo convierte en un PDF listo para entregar. Hay dos tipos de salida:

- Documentos en tamaño A4: manuales, informes, propuestas, especificaciones, guías, apuntes o trabajos académicos. Incluyen portada, índice con números de página, numeración en el pie de cada página con el total (por ejemplo, "Página 3 de 12"), encabezados por capítulo y figuras y tablas numeradas. El diseño funciona igual en un documento de 2 páginas que en uno de 200.
- Presentaciones en formato 16:9 (1280×720), con fondo oscuro, tarjetas con iconos y varios tipos de diapositiva.

El generador no se conecta a ningún servicio de inteligencia artificial ni necesita claves de API. Todo el contenido está en los `.md`, que se pueden leer y corregir antes de generar el PDF.

Cómo funciona

1. Cada archivo de `content/` empieza con un frontmatter: un bloque entre dos líneas `---` que indica si es un documento o una presentación, el título, el autor y otros datos.
2. El Markdown se convierte en un modelo interno con los títulos, párrafos, listas, tablas o diapositivas.
3. Ese modelo se combina con una plantilla HTML y una hoja de estilos CSS.
4. Un motor de impresión convierte el resultado en PDF.
5. El PDF se vuelve a abrir para comprobar que es válido, contar sus páginas y, en las presentaciones, verificar que todas las páginas tienen proporción 16:9.

Motores de impresión

Cada tipo de salida usa el motor que mejor resuelve sus necesidades:

	Documentos	**Presentaciones**
Motor	WeasyPrint	Google Chrome en modo headless
Necesidades	Índice con números de página, numeración con el total de páginas, encabezados por capítulo, títulos que no quedan aislados al final de una página	Degradados, desenfoque, flexbox, fuentes web, iconos SVG
Motivo	WeasyPrint implementa el CSS para medios paginados (`target-counter`, `leader`, `string-set`). Chrome no puede numerar un índice	Chrome dibuja igual que un navegador. WeasyPrint no soporta `filter: blur()`

Ambos motores están detrás de la misma interfaz, por lo que se puede reemplazar uno sin modificar la lógica del programa.

Requisitos

Requisito	Versión	Uso
Python	3.12 o superior (probado con 3.14)	Ejecutar el generador
Google Chrome, Microsoft Edge o Chromium	Reciente	Generar las presentaciones
Pango	La que instale el gestor de paquetes	Librería de texto que usa WeasyPrint (paso 4 de la instalación)

Las librerías de Python se instalan en el paso 3:

Librería	Uso
`weasyprint`	Genera los documentos en PDF
`markdown-it-py`	Interpreta el Markdown
`pyyaml`	Lee el frontmatter
`jinja2`	Combina el contenido con las plantillas HTML
`pypdf`	Revisa el PDF generado
`pytest`	Ejecuta las pruebas automáticas (solo desarrollo)
`ruff`	Revisa el estilo y los errores del código (solo desarrollo)

Instalación

1. Clonar el repositorio

```bash
git clone https://github.com/S4nti4goCoder/genpdf.git
cd genpdf
```

Todos los comandos siguientes se ejecutan dentro de la carpeta `genpdf`.

2. Crear y activar el entorno virtual

Un entorno virtual (`.venv`) es una carpeta con su propio Python y sus propias librerías, aislada del resto del sistema. Cumple la misma función que `node_modules` en un proyecto de Node.

En Windows (PowerShell):

```powershell
python -m venv .venv
.venv\Scripts\Activate.ps1
```

En macOS y Linux:

```bash
python3 -m venv .venv
source .venv/bin/activate
```

Con el entorno activado, la terminal muestra `(.venv)` al inicio de la línea y el comando `python` usa el Python del proyecto. Hay que activarlo cada vez que se abre una terminal nueva.

3. Instalar las librerías de Python

```bash
python -m pip install --upgrade pip
python -m pip install -r requirements-dev.txt
```

Las versiones están fijadas en `requirements.txt` y `requirements-dev.txt`, así que todas las personas del equipo instalan exactamente las mismas.

4. Instalar Pango

WeasyPrint dibuja el texto con Pango, una librería del sistema que no se instala con pip. Si falta, la generación de documentos falla con el error `cannot load library 'libgobject-2.0-0'`. Los comandos siguientes son los que indica la documentación oficial de WeasyPrint.

En Windows se instala con MSYS2:

1. Descargar el instalador desde https://www.msys2.org/. El archivo `msys2-x86_64-....exe` sirve para la mayoría de los equipos; el de ARM64 solo para procesadores ARM.
2. Instalarlo con las opciones por defecto, en `C:\msys64`. Al final, desmarcar "Run MSYS2 now".
3. Instalar Pango desde PowerShell:

```powershell
$env:MSYSTEM = 'UCRT64'
C:\msys64\usr\bin\bash.exe -lc "pacman -S --noconfirm --needed mingw-w64-ucrt-x86_64-pango"
```

El generador busca Pango en `C:\msys64\ucrt64\bin`. Si MSYS2 se instaló en otra carpeta, hay que definir la variable `WEASYPRINT_DLL_DIRECTORIES` con la ruta de su carpeta `ucrt64\bin`.

En macOS:

```bash
brew install weasyprint
```

En Linux:

Distribución	Comando
Ubuntu	`sudo apt install libpango-1.0-0 libharfbuzz0b libpangoft2-1.0-0 libharfbuzz-subset0`
Debian	`sudo apt install libpango-1.0-0 libpangoft2-1.0-0 libharfbuzz-subset0`
Fedora	`sudo dnf install pango`
Arch Linux	`sudo pacman -S pango`

5. Verificar la instalación

```BASH
python -m pytest
```

Todas las pruebas deben pasar. Si no hay un navegador basado en Chromium instalado, la prueba de presentaciones aparece como `skipped` (omitida) y no como fallida.

Uso

Los comandos se ejecutan desde la carpeta del proyecto, con el entorno virtual activado.

Comandos

Los comandos `build` y `check` reciben el nombre de un archivo de `content/` sin la extensión `.md`. Por ejemplo, para el archivo `content/mi-cv.md` el nombre es `mi-cv`.

Comando	Resultado
`python -m genpdf build`	Genera el PDF de todos los archivos de `content/`
`python -m genpdf build mi-cv`	Genera solo `content/mi-cv.md` y lo guarda como `output/mi-cv.pdf`
`python -m genpdf build mi-cv documento-bajas`	Genera varios archivos a la vez
`python -m genpdf check`	Revisa todos los archivos sin generar nada
`python -m genpdf check documento-bajas`	Revisa solo `content/documento-bajas.md`
`python -m genpdf new doc mi-cv`	Crea `content/mi-cv.md` con la plantilla de documento
`python -m genpdf new deck charla-git`	Crea `content/charla-git.md` con la plantilla de presentación
`python -m genpdf --help`	Muestra la ayuda

Si el nombre no existe, el comando lo indica y muestra los archivos disponibles.

Opciones

Opción	Comandos	Efecto
`-v`, `--verbose`	todos	Muestra el tiempo de cada archivo, su origen y las advertencias del motor
`--content CARPETA`	todos	Carpeta de los `.md` (por defecto, `content`)
`--out CARPETA`	`build`	Carpeta de los PDF (por defecto, `output`)

Resultado del comando

`build` muestra una línea por archivo con la ruta del PDF, su peso y su número de páginas:

```text
OK    output/<nombre>.pdf  <peso> KB  <páginas> pages
```

El comando termina con código 0 si todo salió bien y con código 1 si algún archivo falló o no cumplió una regla, lo que permite usarlo en scripts y en integración continua.

Flujo de trabajo

1. Crear el archivo: `python -m genpdf new doc informe-mensual`.
2. Escribir el contenido en `content/informe-mensual.md`.
3. Revisarlo: `python -m genpdf check informe-mensual`.
4. Generar el PDF: `python -m genpdf build informe-mensual`.
5. Abrir `output/informe-mensual.pdf`, corregir el `.md` si hace falta y repetir el paso 4.

Escribir un documento

Frontmatter

Es el bloque del principio del archivo, entre dos líneas `---`. Solo `type` y `title` son obligatorios.

```yaml
---
type: document
title: Manual de instalación y despliegue
subtitle: Versión 2.1
author: Nombre Apellido
date: 2026-10-22
cover: true
toc: true
---
```

Campo	Obligatorio	Valor por defecto	Descripción
`type`	Sí		`document` o `deck`
`title`	Sí		Título. Aparece en la portada y en el encabezado de cada página
`subtitle`	No	vacío	Subtítulo
`author`	No	vacío	Autor
`date`	No	vacío	Fecha. Se muestra tal como se escribe
`cover`	No	`false`	Portada a página completa, sin número de página
`toc`	No	`false`	Índice con números de página. Requiere al menos un título `#`
`tocDepth`	No	`2`	Niveles de título que se incluyen en el índice
`maxPages`	No	sin límite	Número máximo de páginas. Si el PDF lo supera, el comando termina con error
`lang`	No	`es`	Idioma del texto, usado para dividir palabras en sílabas
`theme`	No	`default`	Solo se aplica a las presentaciones

Un campo mal escrito, como `maxpages` en lugar de `maxPages`, produce un error con la lista de campos válidos.

Bloques disponibles

Bloque	Sintaxis	Resultado en el PDF
Capítulo	`# Título`	Empieza en una página nueva y aparece en el índice y en el encabezado
Secciones	`##`, `###`, `####`	No quedan aisladas al final de una página
Negrita, cursiva, tachado	`**texto**`, `*texto*`, `~~texto~~`	Formato dentro del párrafo
Código en línea	`` `código` ``	Fuente monoespaciada con fondo gris
Enlace	`[texto](https://...)`	Enlace que se puede pulsar
Lista con viñetas	`- elemento`	Se anida con dos espacios de sangría
Lista numerada	`1. elemento`	Puede empezar en cualquier número
Tabla	Columnas separadas con `|` y una fila de guiones bajo el encabezado	El encabezado se repite si la tabla continúa en otra página
Pie de tabla	Un párrafo `Tabla: texto` justo antes de la tabla	"Tabla N. texto", con numeración automática
Bloque de código	Tres comillas invertidas seguidas del lenguaje	Fondo gris y etiqueta con el lenguaje
Imagen con pie	`![texto alternativo](img/x.png "pie")`	"Figura N. pie", ajustada al ancho de la página
Cita	`> texto`	Recuadro con borde de color
Separador	Una línea con `---`	Línea horizontal
Salto de página	`<!-- pagebreak -->`	La página termina en ese punto
HTML	Cualquier etiqueta HTML	Se incluye sin cambios en el PDF
Comentario	`<!-- texto -->`	No aparece en el PDF

El archivo `content/templates/document.md` contiene un ejemplo comentado de cada bloque.

Las imágenes se enlazan con una ruta relativa al `.md`. Lo habitual es guardarlas en una carpeta `img/` junto al archivo, por ejemplo `content/img/`. Se admiten los formatos PNG, JPG y SVG.

Escribir una presentación

Frontmatter

```yaml
---
type: deck
title: Introducción a Git
author: Nombre Apellido
date: 2026-10-22
theme: github
---
```

Hay dos temas: `github`, con la paleta del modo oscuro de GitHub, y `default`, en tonos índigo con acentos turquesa y rosa.

Reglas generales

- Una línea que solo contiene `---` separa una diapositiva de la siguiente. Dentro de un bloque de código no tiene ese efecto.
- `<!-- slide: tipo -->` define el tipo de diapositiva. Si se omite, la primera diapositiva es `cover` y las demás son `bullets`.
- Un párrafo corto escrito justo antes del título (`#` o `##`) se muestra como antetítulo: un texto pequeño en mayúsculas y en color de acento.
- Todas las diapositivas, excepto la portada, muestran su número en la esquina inferior derecha.

Tipos de diapositiva

Tipo	Uso	Estructura
`cover`	Portada	Antetítulo, título `#` y un párrafo de subtítulo. El autor y la fecha se toman del frontmatter
`cards`	Conceptos en tarjetas	Cada `###` crea una tarjeta. `<!-- icon: nombre -->` antes del `###` le asigna un icono
`bullets`	Lista de ideas	Una lista con viñetas o numerada
`two-columns`	Comparación	`<!-- column -->` separa las dos columnas. Cada una admite `###`, icono, lista o código
`quote`	Cita destacada	`> cita` seguida de un párrafo con el autor

Tipo	Uso	Estructura
`full-image`	Diagrama o imagen	Una imagen. Con título se muestra enmarcada; sin título ocupa toda la diapositiva
`closing`	Cierre	Título `#`, una frase y una lista que se muestra como etiquetas

Límites de contenido

Para que el contenido no se salga de la diapositiva:

Tipo	Máximo
`bullets`	6 viñetas, incluidas las anidadas
`cards`	4 tarjetas, con un máximo de 3 viñetas cada una
`two-columns`	6 viñetas por columna

`check` avisa cuando una diapositiva supera su límite. Los valores se definen en `SLIDE_CAPACITY`, dentro de `genpdf/domain/rules.py`.

Iconos

`alert`, `arrow-right`, `book`, `chart`, `check`, `clock`, `cloud`, `code`, `database`, `download`, `eye`, `file`, `folder`, `git-branch`, `git-commit`, `git-fork`, `git-merge`, `git-pull-request`, `github`, `globe`, `heart`, `history`, `issue`, `layers`, `lightbulb`, `lock`, `message`, `package`, `play`, `rocket`, `search`, `server`, `settings`, `shield`, `sliders`, `star`, `tag`, `target`, `terminal`, `upload`, `user`, `users`, `zap`.

Para añadir un icono basta con copiar un archivo SVG de 24×24 en `genpdf/presentation/icons/`. El nombre del archivo, sin la extensión, es el nombre del icono.

El archivo `content/templates/deck.md` contiene un ejemplo de cada tipo de diapositiva.

Plantillas

Ninguna plantilla es obligatoria: cualquier `.md` con frontmatter guardado en `content/` se puede convertir en PDF. Las plantillas existen para no empezar de cero y son de dos tipos.

Plantillas de contenido

Son archivos `.md` de ejemplo. Las del proyecto están en `content/templates/`:

Archivo	Contenido
`document.md`	Un ejemplo comentado de cada bloque de un documento
`deck.md`	Un ejemplo de cada tipo de diapositiva
`img/`	La imagen que usan los dos ejemplos

`build` solo genera los archivos que están directamente en `content/`, así que las plantillas no se generan junto con el resto. Para ver cómo quedan en PDF:

```bash
python -m genpdf build document deck --content content/templates
```

Se pueden crear plantillas propias para los documentos que se repiten, como una hoja de vida, un informe mensual o un acta de reunión. Basta con guardar el `.md` con la estructura deseada en `content/templates/` y copiarlo a `content/` con otro nombre cada vez que se use.

Plantillas de diseño

Definen el aspecto del PDF y están en `genpdf/presentation/`. No hace falta saber Python para modificarlas:

Qué se quiere cambiar	Dónde
Colores y fuentes de una presentación	Un tema nuevo en `themes/` (ver *Añadir un tema*)
Márgenes, tipografía o colores de los documentos	`styles/document.css`
Diseño de las diapositivas	`styles/deck.css`
Estructura de la portada o del índice	`templates/document.html`
Iconos disponibles	`icons/`

Crear un tipo de diapositiva o un bloque nuevo sí requiere modificar el código. La sección *Cómo extender el proyecto* lo explica paso a paso.

Validaciones

Antes de generar: `check`

- El frontmatter tiene los campos obligatorios y cada campo tiene el tipo correcto.
- No queda texto provisional: `lorem ipsum` o las palabras `TODO` y `XXX` en mayúsculas. Se ignoran dentro de los bloques de código, y "todo" en minúsculas no se considera un error.
- Todas las imágenes enlazadas existen.
- No hay saltos de nivel entre títulos, como un `###` directamente bajo un `#`.
- Si `toc` es `true`, existe al menos un título `#`.
- En las presentaciones, ninguna diapositiva supera su límite y el tema y los iconos existen.

Cada problema se informa con el archivo y el número de línea.

Después de generar: `build`

- El PDF no está vacío y empieza con la firma `%PDF`.
- Se cuentan las páginas del archivo generado.
- Si el frontmatter define `maxPages` y se supera, se informa y el comando termina con código 1.
- En las presentaciones, cada página mide 960×540 puntos, que equivale a 16:9.

Estructura del proyecto

```text
genpdf/
  content/               archivos .md que se convierten en PDF
    templates/           plantillas de contenido de referencia
  docs/                  esta documentación en PDF
  output/                PDF generados (no se suben al repositorio)
  genpdf/                código del generador
    domain/              modelos y reglas
    application/         casos de uso e interfaces
    infrastructure/      integración con las librerías externas
    presentation/        plantillas, estilos, temas e iconos
    cli/                 comandos de terminal
  tests/                 pruebas automáticas
  README.md              este documento
  pyproject.toml         configuración del proyecto, de ruff y de pytest
  requirements.txt       librerías necesarias para generar PDF
  requirements-dev.txt   librerías anteriores más las de desarrollo
  .gitignore             archivos que el repositorio ignora
```

Las carpetas `.venv/`, `__pycache__/`, `.pytest_cache/` y `.ruff_cache/` se crean automáticamente al instalar o ejecutar el proyecto. No forman parte del código y se pueden borrar en cualquier momento.

Carpeta `content`

Aquí se guardan los `.md` que se convierten en PDF. Cada archivo genera `output/<nombre>.pdf`. En el repositorio solo contiene la subcarpeta `templates/`, con las plantillas de contenido descritas en la sección *Plantillas*.

Carpeta `docs`

Archivo	Contenido
`README.pdf`	Esta documentación en PDF, con portada e índice

Carpeta `genpdf`

El código está organizado en capas. Una capa interior nunca depende de una exterior: el dominio no conoce WeasyPrint y los casos de uso no saben qué motor genera el PDF.

Archivo	Función
`__init__.py`	Convierte la carpeta en un paquete de Python
`__main__.py`	Punto de entrada de `python -m genpdf`

Capa `domain`

Contiene las reglas del programa y no depende de ninguna librería externa.

Archivo	Función
`blocks.py`	Bloques de contenido (título, párrafo, lista, tabla, código, imagen, cita, separador, salto de página, HTML) y formato dentro del texto (negrita, cursiva, enlace, código en línea)
`metadata.py`	Campos del frontmatter, valores por defecto y validación de tipos
`models.py`	`Document`, `Deck`, `Slide`, tipos de diapositiva y `Theme`
`rules.py`	Reglas de validación: texto provisional, saltos de título, índice sin capítulos, límites de diapositivas, temas e iconos
`errors.py`	Errores propios (`ContentError`, `RenderError`, `ValidationError`), con archivo y línea

Capa `application`

Contiene los casos de uso, es decir, las operaciones que ofrece el programa.

Archivo	Función
`ports/`	Interfaces que el programa necesita del exterior: leer Markdown, generar PDF, leer y escribir archivos, revisar un PDF
`build_content.py`	Lee un `.md`, identifica si es documento o presentación y lo envía al caso de uso correspondiente
`build_document.py`	Genera un documento: valida, imprime, guarda y verifica
`build_deck.py`	Genera una presentación y comprueba la proporción 16:9 de cada página
`validate_content.py`	Comando `check`: aplica todas las reglas sin generar nada
`create_content.py`	Comando `new`: crea un `.md` a partir de una plantilla y nunca sobrescribe uno existente
`guards.py`	Comprobaciones comunes: validez del PDF, existencia de imágenes y límite `maxPages`

Capa `infrastructure`

Implementa las interfaces usando las librerías externas.

Archivo	Función
`frontmatter.py`	Separa el bloque YAML del inicio y lo interpreta con pyyaml
`markdown_parser.py`	Punto de entrada de la lectura: devuelve un `Document` o un `Deck`
`markdown_blocks.py`	Convierte el resultado de markdown-it en bloques en una sola pasada
`markdown_inline.py`	Convierte el formato dentro del texto: negrita, enlaces, código en línea
`deck_assembler.py`	Divide una presentación en diapositivas y organiza antetítulo, tarjetas y columnas
`templating.py`	Configura Jinja2 y da acceso a plantillas, estilos, temas e iconos
`weasyprint_renderer.py`	Genera los documentos con WeasyPrint y localiza Pango en Windows
`chrome_renderer.py`	Genera las presentaciones con Chrome. Lo busca en las rutas habituales, usa Edge como alternativa y acepta la variable `CHROME_PATH`
`pdf_inspector.py`	Abre el PDF con pypdf para contar y medir sus páginas
`file_storage.py`	Lectura y escritura de archivos

Capa `presentation`

Contiene el diseño en archivos HTML y CSS, sin código Python. Cualquier persona con conocimientos de diseño web puede modificar el aspecto de los PDF desde aquí.

Archivo	Función
`templates/document.html`	Estructura del documento: portada, índice y cuerpo
`templates/deck.html`	Estructura de cada diapositiva: fondo, pie y número
`templates/_blocks.html`	Representación HTML de cada bloque, compartida por documentos y presentaciones
`templates/_slides.html`	Representación HTML de cada tipo de diapositiva
`styles/document.css`	Diseño A4: márgenes, numeración, portada, índice, tablas y figuras
`styles/deck.css`	Diseño de las diapositivas, con los colores tomados del tema
`themes/github.css`, `themes/default.css`	Temas de presentación: solo definen colores y fuentes
`icons/*.svg`	Iconos para tarjetas y columnas
`starters/*.md`	Plantillas que usa el comando `new`

Capa `cli`

Archivo	Función
`main.py`	Interpreta los comandos `build`, `check` y `new`, ejecuta el caso de uso y muestra el resultado
`container.py`	Asocia cada interfaz con su implementación. Es el único archivo que conoce qué motor genera cada tipo de PDF

Carpeta `tests`

Archivo	Cobertura
`conftest.py`	Datos compartidos entre pruebas (fixtures de pytest)
`helpers.py`	Utilidades para crear frontmatter y un almacenamiento en memoria
`test_markdown_parser.py`	Todos los bloques y casos especiales: listas anidadas, tablas con celdas vacías, código con comillas invertidas
`test_frontmatter.py`	Campos faltantes, tipos incorrectos, valores por defecto y campos mal escritos
`test_deck_parser.py`	Separación de diapositivas, tipos, antetítulo, tarjetas, columnas e imágenes
`test_rules.py`	Cada regla de validación
`test_use_cases.py`	Validación y creación de archivos
`test_integration.py`	Generación de PDF reales con WeasyPrint y Chrome, revisados con pypdf

Desarrollo y extensión

Comandos de desarrollo

```bash
ruff check .        # revisa el código
python -m pytest    # ejecuta las pruebas
```

Cómo extender el proyecto

Gracias a la separación en capas, cada ampliación toca pocos archivos y siempre los mismos. Las rutas de esta sección son relativas a la carpeta `genpdf/` del código.

Añadir un bloque nuevo

Por ejemplo, un recuadro de aviso.

1. En `domain/blocks.py`, crear una clase con `kind = "aviso"` y el campo `line`, y añadirla a la unión `Block`. Si el bloque contiene otros bloques, incluirlo también en la función `walk()`.
2. En la capa `infrastructure`, hacer que el lector lo produzca. Hay dos formas:
   - si corresponde a un elemento de Markdown, añadir un manejador en `TokenReader._handlers`, en `markdown_blocks.py`;
   - si se escribe con un comentario como `<!-- aviso -->`, tratarlo en `markdown_parser.py` de la misma forma que `pagebreak`.
3. En `presentation/templates/_blocks.html`, añadir una rama `{% elif b.kind == "aviso" %}` a la macro `block()` con su HTML.
4. Darle estilo en `presentation/styles/document.css` y, si puede aparecer en las presentaciones, también en `presentation/styles/deck.css`.
5. Si contiene texto escrito por el autor, incluirlo en la función `_prose()` de `domain/rules.py` para que `check` lo revise.
6. Añadir una prueba en `tests/test_markdown_parser.py`.

Añadir un tipo de diapositiva

Por ejemplo, una línea de tiempo.

1. En `domain/models.py`, añadir `TIMELINE = "timeline"` a `SlideKind`.

2. Si necesita una estructura propia, crear una función en `infrastructure/deck_assembler.py` y registrarla en `_BUILDERS`. Si no, los bloques se conservan tal como están escritos.
3. En `presentation/templates/_slides.html`, crear una macro llamada `timeline`. La plantilla la encuentra por el nombre del tipo; los guiones del nombre se escriben como guiones bajos.
4. Darle estilo con la clase `.slide--timeline` en `presentation/styles/deck.css`.
5. Si tiene un límite de contenido, añadirlo a `SLIDE_CAPACITY` en `domain/rules.py`.
6. Añadir un ejemplo en `content/templates/deck.md`.

Añadir un tema

1. Copiar `presentation/themes/github.css` con el nombre del tema nuevo, por ejemplo `presentation/themes/corporativo.css`.
2. Cambiar los valores de las variables. Los temas solo definen colores, resplandores y fuentes; la estructura vive en `deck.css`.
3. Usarlo con `theme: corporativo` en el frontmatter. `check` y `build` lo reconocen sin más cambios.

Para usar otra fuente de Google Fonts, se añade un `@import` al principio del archivo del tema y se cambia la variable `--font-sans`.

Añadir un icono

Copiar un archivo SVG de 24×24 que use `stroke="currentColor"` en `presentation/icons/`. El nombre del archivo, sin la extensión, es el nombre que se usa en `<!-- icon: nombre -->`.

Cambiar un motor de impresión

Los casos de uso solo conocen la interfaz `PdfRenderer`. Para usar otro motor se crea una clase nueva en `infrastructure/` con el método `render(content, theme, base_dir) -> bytes` y se cambia una sola línea en `cli/container.py`.

Configuración y solución de problemas

Variables de entorno

Variable	Uso	Cuándo definirla
`CHROME_PATH`	Ruta completa al ejecutable de Chrome	Si Chrome está instalado en una ruta no habitual
`WEASYPRINT_DLL_DIRECTORIES`	Carpeta con las librerías de Pango	En Windows, si MSYS2 no está en `C:\msys64`

Solución de problemas

Mensaje o síntoma	Causa	Solución
`WeasyPrint cannot load Pango` o `cannot load library 'libgobject-2.0-0'`	Pango no está instalado o no se encuentra	Repetir el paso 4 de la instalación o definir `WEASYPRINT_DLL_DIRECTORIES`
`Chrome was not found`	No hay Chrome, Edge ni Chromium en las rutas habituales	Instalar Chrome o definir `CHROME_PATH`
`cannot write the PDF`	El PDF está abierto en un visor que lo bloquea	Cerrar el PDF y volver a generarlo
`running scripts is disabled on this system` al activar el entorno en Windows	PowerShell bloquea la ejecución de scripts	Ejecutar una vez `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
`No module named genpdf`	El comando no se ejecutó desde la carpeta del proyecto	Abrir la terminal en la carpeta `genpdf`
`No module named weasyprint`	El entorno virtual no está activado	Activarlo como se indica en el paso 2
`unknown frontmatter field`	Un campo del frontmatter está mal escrito	Corregirlo según la lista de campos válidos que muestra el mensaje

Limitaciones conocidas

- Las presentaciones pesan entre 2 y 3 MB, porque Chrome guarda los efectos de desenfoque como imágenes.

- Las fuentes de las presentaciones (Inter y JetBrains Mono) se descargan de Google Fonts al generar. Sin conexión se usan las fuentes del sistema. Los documentos solo usan fuentes del sistema.
- Los documentos tienen un único diseño. El campo `theme` solo se aplica a las presentaciones.
- Los bloques de código no tienen resaltado de sintaxis por colores.

Autor



S4nti4goCoder

Desarrollador de software

PORTFOLIO	santiagocoder.vercel.app
GITHUB	github.com/S4nti4goCoder
REPOSITORIO	github.com/S4nti4goCoder/genpdf